NVIDIA DGX Spark Configuration Log

This log details the configuration process of our **NVIDIA DGX Spark** unit. The device is an ARM-based desktop AI system, which necessitates specific software versions (aarch64/arm64) distinct from standard x86 desktops. The setup is divided into two phases: the initial system initialization via the local hotspot and the subsequent software environment configuration.

1. Phase 1: Initial Hardware Setup & First Boot

System Specifications

Device: NVIDIA DGX Spark

Processor (SoC): NVIDIA GB10 Grace Blackwell Superchip

Architecture: ARM64 (aarch64)

- **CPU:** 20-Core Custom Arm Processor
  - **Performance Cores:** 10x Arm Cortex-X925 (up to 4.0 GHz)
  - **Efficiency Cores:** 10x Arm Cortex-A725 (up to 2.8 GHz)
- **GPU:** NVIDIA Blackwell Architecture
  - **AI Performance:** Up to 1 PetaFLOP (FP4 Sparse) / 500 TFLOPS (FP4 Dense)
  - **Cores:** 5th Generation Tensor Cores, 4th Generation RT Cores
- **Memory (RAM):** 128 GB Unified System Memory
  - **Type:** LPDDR5x
  - **Bandwidth:** 273 GB/s (Shared coherently between CPU and GPU)
  - **Bus Width:** 256-bit
- **Storage:** 4 TB NVMe M.2 SSD (PCIe Gen4, Self-Encrypting)
- **Networking:**
  - **LAN:** 1x 10 Gigabit Ethernet (10GbE RJ-45)
  - **Interconnect:** NVIDIA ConnectX-7 SmartNIC (2x QSFP ports for clustering/high-speed transfer)
  - **Wireless:** Wi-Fi 7 (802.11be), Bluetooth 5.4
- **I/O Ports:** 4x USB-C (USB 4/Thunderbolt compatible, 1x supports 240W PD power input), 1x HDMI 2.1a

1.1. Prerequisites & Physical Connection

Since we configured the device without a dedicated monitor initially, we utilized the NVIDIA First-Boot Wizard via the device's self-hosted hotspot.

1. **Power On:** We switched on the DGX Spark.
2. **Hotspot Connection:** Upon boot, the system broadcasted a temporary WiFi network for configuration.

    - **SSID:** `spark-<ID>` (Found on the Quick Start Guide cover).
    - **Password:** [Refer to the sticker on the guide].
3. **Accessing the Wizard:**
    - We connected a laptop to the `spark-<ID>` WiFi network.
    - In a browser, we navigated to **http://spark-<ID>.local** (or `http://device-name.local`).
4. **Configuration Steps:**
    - **EULA:** Accepted the NVIDIA Terms of Use.
    - **Localization:** Set Language to English, Keyboard to US, and Timezone example: `America/New York`.
    - **User Account:** We created the primary administrator account.
        - *Policy:* We purposefully avoided reserved usernames such as `admin`, `root`, or `sysadmin` to prevent conflict with OS defaults.
    - **Network Bridge:** We selected our local WiFi network from the list to allow the device to pull updates.
5. **Completion:** After connecting to the local WiFi, the device performed an automatic self-update and rebooted.

Note: We couldn't connect to the Open Michigan Tech network due to (requiring certificates) or captive portals (terms acceptance pages) during the initial headless setup.

2. Phase 2: System Configuration & Environment Setup

Once the device finished downloading updates via the setup wizard and rebooted, we connected it to a monitor and used the system's local user interface to proceed with software provisioning via the terminal.

2.1. System Updates

First, we ensured the base OS and kernel were fully patched. We opened the terminal and ran the following commands sequentially:

Update package lists and upgrade installed packages: `sudo apt update && sudo apt upgrade -y`

Handle distribution upgrades (Kernel/Firmware updates): `sudo apt dist-upgrade -y`

Refresh Snap packages: `sudo snap refresh`

2.2. SSH Remote Access

To enable remote management from our workstations:

1. **Identify IP Address:** We retrieved the IP using the command `ifconfig` or by checking the router's client list.
2. **Connection:** We verified access via SSH using the account created in Phase 1.
    - Command: `ssh username@spark-6s23.local` (or `ssh username@<IP_ADDRESS>`)

2.3. Development Environment Setup

Anaconda (Python Environment)

We installed Anaconda to manage our data science libraries.

- *Note:* We specifically used the **aarch64** installer for the device's ARM architecture.

1. **Download Anaconda3-2025.06-0-Linux-aarch64.sh:** `wget https://repo.anaconda.com/archive/Anaconda3-2025.06-0-Linux-aarch64.sh`
2. **Running the installer:** `bash Anaconda3-2025.06-0-Linux-aarch64.sh`
3. **Initialize Anaconda:** The installer will likely ask to initialize Anaconda. Confirm this to add Anaconda to your system's PATH.
4. **Refresh Terminal:** Close and reopen your terminal, or run `source ~/.bashrc` to apply the changes to your current terminal session.
5. **Verification:** Launched the GUI by running `anaconda-navigator`.

Note, if the terminal defaults for Anaconda terminal, it can always be turned off using

`conda init --reverse $SHELL`

Visual Studio Code

We installed the Visual Studio Code IDE.

1. **Download:** We navigated to the directory where we downloaded the `.deb` file (ensure you download the **Arm64** version for this device, not amd64). `wget https://vscode.download.prss.microsoft.com/dbazure/download/stable/7d842fb85a0275a4a8e4d7e040d2625abbf7f084/code_1.105.1-1760482543_amd64.deb`
2. **Install:** `sudo apt install ./https://vscode.download.prss.microsoft.com/dbazure/download/stable/7d842fb85a0275a4a8e4d7e040d2625abbf7f084/code_1.105.1-1760482543_amd64.deb`
3. **Dependency Fix:** If dependency errors occurred, we fixed them by running: `sudo apt-get install -f`

GitHub Desktop

We set up the GitHub Desktop client for version control.

1. **Download the package:** `wget https://github.com/shiftkey/desktop/releases/download/release-3.4.3-linux1/GitHubDesktop-linux-arm64-3.4.3-linux1.deb`
2. **Install the package:** `sudo apt install ./GitHubDesktop-linux-arm64-3.4.3-linux1.deb`

Note!, this will not install git, git needs to be installed separately using "sudo apt install git"

2.4. Remote Desktop (XRDP) Configuration

To allow graphical remote access from Windows/Mac clients, we set up XRDP with the XFCE desktop environment.

1. Install Desktop Environment & XRDP: We installed XFCE (lighter weight than GNOME) and the RDP server: `sudo apt install xfce4 xfce4-goodies -y` `sudo apt install xrdp -y`

2. SSL Certificate Permissions: We granted the `xrdp` user access to system certificates to prevent connection errors: `sudo adduser xrdp ssl-cert`

3. D-Bus / Session Isolation Fix (Critical): We encountered an issue where the remote session would conflict with the local console (causing a black screen). To fix this, we edited the startup script:

Run this command in terminal: `sudo nano /etc/xrdp/startwm.sh`
Then copy and paste the following:

```
#!/bin/sh

# Load system locale settings
if [ -r /etc/default/locale ]; then
  . /etc/default/locale
  export LANG LANGUAGE
fi

# --- CRITICAL: SESSION ISOLATION ---
# These lines prevent the remote session from latching onto
# the local machine's D-Bus, which fixes the "wrong screen" issue.
unset DBUS_SESSION_BUS_ADDRESS
unset SESSION_MANAGER
```

```
# ---------------------------------

# Start XFCE Desktop

Startxfce4
```

Save and exit: by pressing `Ctrl+O`, `Enter`, then `Ctrl+X`.

4. Network & Firewall: We allowed the RDP port through the firewall: `sudo ufw allow 3389/tcp`

5. Finalize: We restarted the service to apply changes: `sudo systemctl restart xrdp`

Troubleshooting Notes:

- **Lag/No GPU usage:** We installed `xorgxrdp` to bridge the RDP session to the GPU: `sudo apt install xorgxrdp`
- **Black Screen:** If a black screen persisted, we verified the `startwm.sh` edits and checked logs using `cat /var/log/xrdp.log`.

2.5. PyCharm Installation

We installed the JetBrains PyCharm IDE via Snap.

- **For Community Edition:** `sudo snap install pycharm-community --classic`
- **For Professional Edition:** `sudo snap install pycharm-professional --classic`
- **For Educational Edition:** `sudo snap install pycharm-educational --classic`

2.6. SSH Through Visual Studio Code

Step 1 - Install the VS Code "*Remote - SSH*" extension

1. Open VS Code
2. Go to Extensions (left sidebar)
3. Search: Remote - SSH
4. Install it (and also Remote - SSH: Editing Configuration Files)

Step 2 - Configure SSH in VS Code

- Add the host automatically (easiest)

1. Press ⌘ + Shift + P (Mac) or Ctrl + Shift + P (Windows)
2. Search: "Remote-SSH: Add New SSH Host"
3. ssh username@192.168.x.x
4. Select the SSH config file (usually `~/.ssh/config`)
5. Vscode will add something like this
   Host ubuntu-server
     HostName 192.168.x.x
     User username

Step 3 - Connect to the Remote Server

1. Press ⌘ + Shift + P or Ctrl + Shift + P again
2. Search: Remote-SSH: Connect to Host
3. Click your host (ubuntu-server)
4. VS Code opens a new window, now connected to the Ubuntu machine.

Step 4 - Opening Folders & Coding on Remote
Once connected:

1. Click File → Open Folder
2. Pick any directory on the Ubuntu machine
3. VS Code installs its remote server automatically
4. Click Terminal → New Terminal ,This terminal runs inside the Ubuntu machine, not your laptop

2.7. Using Byobu when connecting via SSH

When doing SSH connection we utilized **Byobu**, which is a text-based window manager and terminal multiplexer.

1. **Installation:** We discovered that Byobu was pre-installed on the system. However, if it is not present, it can be installed using: `sudo apt install byobu`

2. **Usage:** To start a session, we simply opened the terminal and ran: `byobu`

3. **Navigation:**Here is a useful reference for the keys shortcuts on how to use Byobu: https://gist.github.com/devhero/7b9a7281db0ac4ba683f

4. Other than having the ability to have multiple terminals open at once, the primary benefit of using Byobu is that it keeps the terminal active in the background, even if the SSH connection is terminated. This is critical for our workflow, as it allows us to run training codes that take hours to complete without maintaining a constant connection to the machine.